

Solo E-Commerce Platform -- Technical Features

Solo E-Commerce Platform

Document Version 1.0 | March 2026

Document Version: 1.1

Date: 09 May 2026

Author: Solo Engineering Team

Status: Final

1. Document Purpose

This document is a technical feature catalogue of the Solo E-Commerce platform -- what is implemented, with which libraries and patterns, on both the React frontend and the NestJS backend, plus database, payments, DevOps and observability. It is meant for engineers onboarding to the codebase, integration partners, and security/audit reviewers.

Companion docs: 01 HLD,
02 LLD,
03 Architecture,
04 Scope & Requirements.

2. Frontend Features (React 19 + Vite 6)

2.1 Stack

Concern	Library / Tool
Framework	React 19
Build	Vite 6
Language	TypeScript 5
Routing	react-router-dom 7 (lazy-loaded admin chunk)
Global state	Zustand 5 (authStore, cartStore, favoritesStore, homeStore)
UI state	React Context (ThemeContext, ToastContext)
Styling	CSS Modules + design tokens (src/styles/tokens.css)
Forms	React Hook Form
Charts	Recharts (<LineChart>, <BarChart>, <PieChart>)
HTTP	Native fetch wrapped by lib/apiClient.ts
Icons	Lucide React
Testing	Vitest 3 (unit), Testing Library (RTL), Playwright (e2e)
Linting	ESLint + Prettier

2.2 Storefront Pages

- Home -- assembled at runtime from /api/content/home sections.
- Product list -- filters (category, brand, designer, price range, in-stock, on-sale, new, featured, best seller), sort, pagination.
- Product detail -- gallery, variant selector, buy box, description tabs, related products.
- Category / Brand / Designer landing -- CMS-managed banners + curated product slots.
- Search -- full-text across product name, SKU, brand, description.

- Cart -- persistent across sessions; live total breakdown.
- Checkout -- address picker -> shipping method -> payment selector

(Stripe Elements / Tabby redirect / Tamara redirect / COD).

- Account -- profile, addresses, orders, returns, loyalty, favorites.
- Auth -- login, register, forgot password, reset password, verify

email.

2.3 Admin Pages (lazy chunk, RBAC-gated)

- Dashboard -- KPI cards + Recharts visualisations (revenue series, top products bar, orders by status).
- Catalog management -- products (CRUD + soft-delete + restore + variant groups + image manager), categories, subcategories, brands, designers, collections.
- Orders -- queue with filters, detail view, state-machine actions, refund.
- Customers -- list, detail, loyalty adjustment.
- CMS -- home page builder, landing pages, banners, navigation, announcements, media library.
- Promo codes -- CRUD with schedule + caps.
- Reports -- sales summary, revenue period, top products, customers LTV, orders by status, low stock; CSV export.
- Audit log -- under "Security" nav group; filter by user, action, entity, date range; pagination.
- Settings -- VAT rate, shipping tiers, loyalty earn rate & redemption cap.
- Bulk orders -- B2B request inbox with quote workflow.

2.4 Cross-cutting Frontend Features

- Token storage -- access JWT in localStorage; refresh via /auth/refresh with one-shot retry on 401.
- Theme -- light/dark/system, persisted via ThemeContext.
- Toasts -- global queue via ToastContext.
- Form validation -- React Hook Form + per-field rules; server-side errors mapped to fields.
- Lazy loading -- admin and other heavy pages via React.lazy().
- Error boundaries -- at route group level; show recoverable UI.
- Code-splitting -- Vite chunks per route + Recharts vendor chunk.
- PWA-ready -- manifest, icons, service-worker hook (deferred at launch).

3. Backend Features (NestJS 10 + Node.js 20)

3.1 Stack

Concern	Library / Tool
Framework	NestJS 10

Runtime	Node.js 20
Language	TypeScript 5
ORM	Prisma 5
Auth	Passport-JWT (HS256) + Argon2id
Validation	class-validator + class-transformer (ValidationPipe global, whitelist + transform + forbidNonWhitelisted)
Logging	nestjs-pino
Tracing	Application Insights SDK
HTTP middleware	Helmet, CORS, compression
Rate limiting	@nestjs/throttler
Image processing	sharp (thumb / medium / large variants)
PDF	pdfkit (invoices)
Email	nodemailer (SMTP)
Payments	stripe, Tabby SDK, Tamara SDK
Storage	@azure/storage-blob
Docs	@nestjs/swagger (/api/docs)
Tests	Jest + Supertest

3.2 Module Catalogue

Auth, Users, Catalog, Products, Categories, Brands, Designers, Cart, Orders, Returns, Promo, Loyalty, Stripe, Tabby, Tamara, CMS, Content, Banners, Navigation, Collections, Announcements, Media, Stock, Admin, Reports, Customers, Settings, BulkOrders, Email, Health, Audit, Common.
(See 02 LLD §2 for the full graph.)

3.3 Authentication & Authorization

- Argon2id password hashing (memory-hard).
- 15-min access JWT + 7-day opaque refresh token (hashed at rest).
- Refresh token rotation with reuse detection -- replay revokes the whole chain.
- JwtAuthGuard, OptionalJwtAuthGuard, RolesGuard, @Roles().
- Email verification + password reset flows with single-use tokens.
- Per-route throttler override: /auth/* = 5 / 15 min / IP; global default 60 / min / IP.

3.4 Catalog

- Products with soft-delete (deletedAt), price snapshot, stock reservation (stockQty / reservedQty), variant grouping (product_groups), images (sharp variants), flags (isNew, isFeatured, isBestSeller).
- Categories + subcategories + brands + designers (CRUD + slug uniqueness + ordering).
- Catalog landing pages with curated slots.

3.5 Cart & Orders

- Server-side total computation on every read (no client trust).
- Atomic order creation: prisma.\$transaction() locks product rows, computes totals, snapshots them onto orders, decrements stock,

posts pending loyalty earn, writes audit, all-or-nothing.

- Order status state machine with order_status_history.
- pdfkit invoice generation with VAT split, TRN, sequential invoice number.

3.6 Payments

Provider	Endpoints	Idempotency	Notes
Stripe	POST /api/stripe/create-payment-intent, POST /api/stripe/webhook	stripe_events PK = event id	Stripe Elements on the SPA
Tabby	POST /api/tabby/checkout, POST /api/tabby/webhook	tabby_events table	redirect to provider
Tamara	POST /api/tamara/checkout, POST /api/tamara/webhook	tamara_events table	redirect to provider
COD	created directly in PROCESSING	n/a	no provider call

PAID transition (Stripe / Tabby / Tamara) fans out inside one

transaction: order status update, stock decrement, stock movement row,

loyalty PENDING -> EARNED, invoice insert, audit log row.

3.7 CMS / Content

Tables: home_page_config, home_page_sections, landing_pages, category_landing_pages, banners, navigation_menu_items, product_collections, announcements, media_assets.

Section types: HERO_BANNER, CATEGORY_TILES, BRAND_STRIP, COLLECTION, EDITORIAL, BEST_SELLERS, NEW_ARRIVALS, PROMO_STRIP, TESTIMONIALS.

Public API: /api/content/* returns assembled payloads optimised for single-round-trip render. Admin writes: /api/admin/{home-page, banners, landing-pages, navigation, collections, announcements, category-landing}.

3.8 Loyalty

- Per-user wallet (loyalty_wallets).
- Ledger (loyalty_transactions): EARNED, REDEEMED, ADJUSTED, EXPIRED, PENDING.
- Earn rate default 2% of subtotalExclVat, configurable in site_settings.loyalty.earnRate.
- Redemption cap default 50% of subtotal, configurable.
- Refund rolls back loyalty earn; admin can post ADJUSTED rows with reason captured in audit.
- Monthly cron expires aged transactions.

3.9 Returns

State machine: REQUESTED -> APPROVED -> IN_TRANSIT -> RECEIVED -> COMPLETED (or REJECTED).

COMPLETED triggers Stripe refund + loyalty rollback inside one transaction; emits audit RETURN_COMPLETED.

3.10 Audit Trail (AuditInterceptor)

- Global APP_INTERCEPTOR.
- Auto-logs every POST/PATCH/PUT/DELETE on /api/admin/*.
- Action name derived from controller method (regex split on capitals -> .toUpperCase()); overrideable via @AuditAction('ORDER_REFUNDED').
- Captures user, email, IP, UA, route, before/after diff in metadata JSONB.
- Auth events written explicitly from AuthService (AUTH_LOGIN_*, AUTH_REGISTERED, AUTH_LOGGED_OUT, AUTH_REFRESH_REUSE_DETECTED).
- audit_logs table is append-only -- no UPDATE/DELETE endpoint.
- Indexes: (createdAt DESC), (userId, createdAt DESC), (action), (entityType, entityId).
- Retention: 90 days hot in PG, nightly cold export, prune.

3.11 Reporting

`/api/admin/reports/{sales-summary, revenue-period, top-products, customers-ltv, orders-by-status, low-stock}`. All accept from, to` and optional categoryId / brandId. CSV export available where practical.

3.12 Admin Dashboard

GET /api/admin/dashboard returns KPI cards (revenue today/7d, orders today, new customers 7d, low-stock count) plus revenue series, top products and recent orders. Recharts on the SPA renders the series.

4. Database Features (PostgreSQL 14 + Prisma 5)

Feature	Detail
Engine	Azure PostgreSQL Flexible Server 14
ORM	Prisma 5; schema.prisma is the single source of truth
Tables	~50 across identity, catalog, cart/orders, returns, payments, CMS, loyalty/promo, stock, ops
JSONB	Used for home_page_sections.config, audit_logs.metadata, flexible CMS payloads
Soft delete	deletedAt tombstones on users, products, categories, brands, etc.
Transactions	prisma.\$transaction() around all critical writes (order create, PAID fan-out, returns)
Migrations	prisma/migrations/ applied via npx prisma migrate deploy on container start
Indexes	See 02 LLD §16.2
Backups	Azure-managed daily snapshots + PITR
Seeding	backend/prisma/seed.ts for dev/demo data

5. Payment Features (in-depth)

5.1 Stripe

- Stripe Elements on the SPA (@stripe/stripe-js + @stripe/react-stripe-js).
- Backend creates a PaymentIntent on order create with metadata { orderId, userId }.
- Webhook signature verified with STRIPE_WEBHOOK_SECRET using stripe.webhooks.constructEvent().
- stripe_events.id PRIMARY KEY guarantees idempotent processing on Stripe retries.
- Handled events: payment_intent.succeeded, payment_intent.payment_failed, charge.refunded, charge.dispute.created.

5.2 Tabby & Tamara (BNPL)

- Provider-hosted checkout: backend creates a session, returns a redirect URL; SPA navigates to it.
- HMAC-signed webhooks verified with TABBY_WEBHOOK_SECRET / TAMARA_NOTIFICATION_TOKEN.
- Per-provider event tables for idempotency.
- Same PAID fan-out as Stripe.

5.3 Cash on Delivery

- No external call.
- Order created in PROCESSING directly with COD payment method.
- Loyalty earn flips to EARNED immediately at create (no pending step).

6. CMS Features

- Drag-and-drop home-page section ordering in the admin SPA.
- Banners with desktop + mobile media URLs and schedule windows (startsAt, endsAt).
- Landing page builder for marketing campaigns (/landing/black-friday-2026).
- Navigation menu tree (self-referencing parentId).
- Curated product collections referenced by COLLECTION section type.
- Top-bar announcements with optional click-through to a promo / landing URL.
- Media library with sharp-generated variants stored in Azure Blob.

7. Loyalty Features (deep dive)

- Earn at 2% of subtotalExcVat (configurable).
- Posted as PENDING at order create; flipped to EARNED at PAID.
- Refund posts a reversing EARNED row (negative net).

- Redemption capped at $\min(\text{walletBalance}, \text{subtotal} \times \text{maxRedeemPct})$, default 50%.
- Admin "Adjust balance" requires reason text -> captured in audit metadata + ledger row.
- Monthly cron expires aged transactions (EXPIRED).
- Account UI shows balance, lifetime earn, paginated ledger.

8. Returns Features

- Requestable from delivered orders only.
- Per-line-item selection (qty + reason).
- Admin approves / rejects with note (audit-logged).
- COMPLETED triggers Stripe refund + loyalty rollback.
- State machine emits per-transition audit events.

9. Admin Dashboard Features (deep dive)

- KPI cards: revenue today, revenue 7d, orders today, new customers 7d, low-stock count.
- Recharts: revenue line series (last 30 d), top-10 products bar, orders-by-status donut.
- Recent orders list with quick state actions.
- Audit Trail page (under "Security" nav group) with rich filters: user, action, entity type/id, date range.
- Reports page exports CSV.

10. DevOps & Infrastructure Features

Feature	Detail
IaC	Bicep (infra/main.bicep + infra/main.parameters.json)
Orchestrator	Azure Developer CLI (azd up, azd deploy)
Hosting	Azure Static Web Apps (frontend) + Azure Container Apps (backend)
Container registry	Azure Container Registry, image pull via User-Assigned Managed Identity (AcrPull)
Database	Azure PostgreSQL Flexible Server 14
Storage	Azure Blob Storage (media container)
Telemetry	Application Insights + Log Analytics
Build commands	npm ci, npm run lint, npm run build, npm run test
Container start	prisma migrate deploy && node dist/main.js (self-healing migration)
CI	GitHub Actions: lint + test + build + docker push + ACA update + SWA deploy
Routine deploy	cd "Solo Website" && azd deploy backend (or frontend, or both)
Rollback	az containerapp revision activate <previousRevision>

11. Monitoring & Observability Features

Signal	Source	Sink
HTTP server traces	App Insights SDK in NestJS	Application Insights
Custom events	AppInsights.trackEvent() for ORDER_PAID, ORDER_REFUNDED, RETURN_COMPLETED, STOCK_LOW	Application Insights
Structured logs	nestjs-pino -> stdout	ACA -> Log Analytics
Browser RUM	App Insights JS snippet (optional)	Application Insights
Container metrics	Built-in ACA metrics	ACA portal + Log Analytics
DB metrics	Built-in PG metrics	Postgres portal
Health probe	/api/health (DB ping + version)	App Insights availability test

Recommended alerts: server-error rate, container restarts, PG CPU/IOPS, webhook failure spikes, availability test failures.

12. Testing Features

Layer	Tool	Scope
Backend unit	Jest	services, guards, interceptors
Backend e2e	Jest + Supertest	controller-level HTTP tests
Frontend unit	Vitest 3 + RTL	hooks, stores, components
Frontend e2e	Playwright	smoke flows: browse -> PDP -> cart -> checkout (test mode)
Static	ESLint + Prettier + tsc --noEmit	both tiers
Coverage targets	>= 70% on backend services & frontend stores/hooks	

CI pipeline (.github/workflows): lint -> test -> build -> docker push -> deploy. Failed checks block merge to main.

End of Technical Features.